



Day 7: Building Your First Backend with PHP

From Static Websites to Dynamic Applications

DAY 7 OF 13

DURATION: 2H 30MIN

Yesterday your webpage **displayed** data. Can it **save** it?

Think about every form you've ever submitted online — a Google Form, a sign-up page, a checkout. Something received that data. Something processed it. Something stored it. That "something" is the **backend**.

The Big Question

Where does submitted information actually go when you hit the submit button?

Pause & Discuss

Turn to your partner. Take 60 seconds. Where do you think the data travels? What receives it? What happens next?

Today's Mission

By the end of this session, you will have **built a working Student Registration System** from scratch — a real backend application that receives form data, validates it, and returns a dynamic confirmation page.



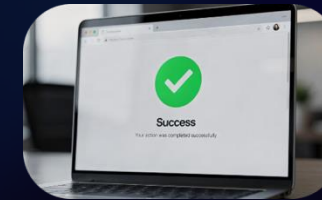
Registration Form

A styled Bootstrap HTML form with student fields



PHP Processing

A backend script that receives and validates POST data



Confirmation Page

A dynamic response page that greets the student by name



Every module today feeds into this single project. Keep your code — you'll need it all.

⚡ Rapid Fire Warm-Up

Before we write our first PHP line, let's fire up the neurons. Discuss each term with your partner — **you have 5 minutes**. No looking at your notes!

JSON

What is it? What does it look like?

Fetch

What does `fetch()` do in JavaScript?

API

What is an API? Name one you've used.

Loop

Write a `for` loop syntax from memory.

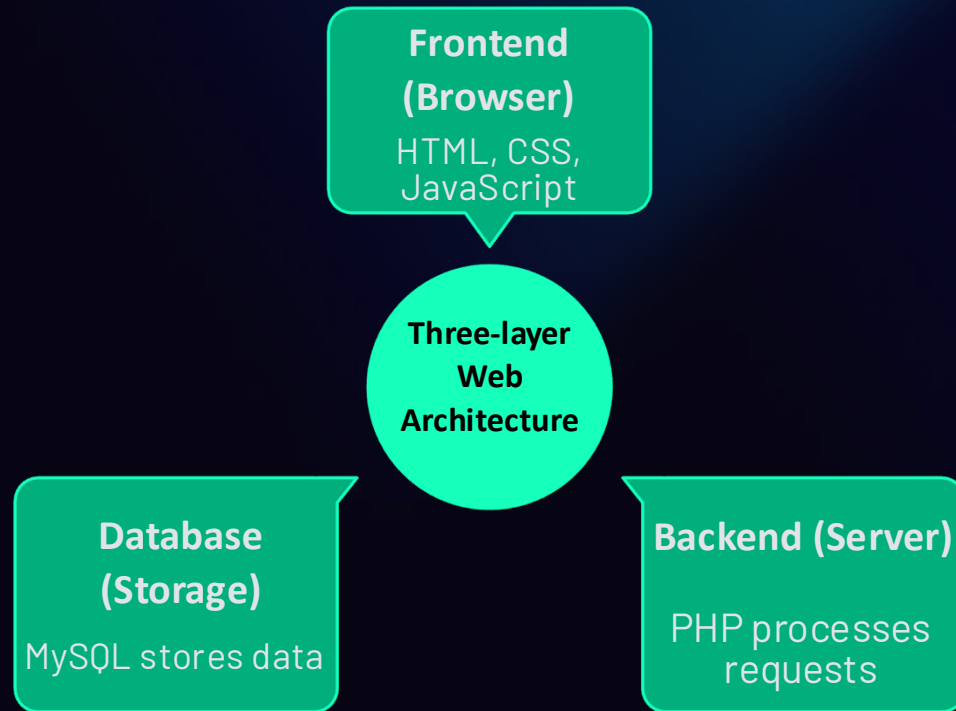
Dynamic HTML

How did you render data into the DOM?

■ 5-MINUTE TIMER

PAIRS DISCUSSION

What Is a Backend?



The **frontend** is everything a user sees — HTML, CSS, JavaScript running in the browser. The **backend** is code running on a server, invisible to the user, that processes requests, runs business logic, and talks to databases. PHP is one of the most widely used backend languages in the world — it powers over 75% of websites including WordPress and Facebook's early codebase.

Key Difference

JavaScript runs *in the browser*. PHP runs *on the server*. The browser never sees your PHP source code — only the output it generates.

Open XAMPP / Laragon. Navigate to your `htdocs` folder. Create `index.php`. Type `<?php echo "Hello World"; ?>`. Open `localhost/index.php` in your browser. You just ran server-side code.

Mission 1 + Challenge

Your Mission

Create a new file called `welcome.php`. Use PHP to print three things on the page: **Your Name**, the **Current Date** using `date("Y-m-d")`, and your **Favourite Programming Language**. Wrap them in proper HTML with a heading and a styled layout.

Code Challenge

Extend your page to also display the **current time** in `H:i:s` format using PHP's `date()` function. Make it update live every time the page loads.

Bonus Round

Display the **visitor's IP address** using PHP's `$_SERVER['REMOTE_ADDR']` superglobal. Add a label like "You are visiting from: 127.0.0.1".

PHP Variables

The Concept

In PHP, every variable starts with a `$` sign. Variables are loosely typed — PHP figures out the type automatically. You can embed PHP variables directly inside HTML to build dynamic pages. This is one of PHP's most powerful features.

```
<?php
$name    = "Priya Sharma";
$college = "MIT Pune";
$branch  = "Computer Engineering";
?>

<h1>Hello, <?= $name ?></h1>
<p><?= $college ?> | <?= $branch ?></p>
```

Live Coding — Follow Along

Create `profile.php`. Declare three PHP variables at the top of the file:

- `$name` — your full name
- `$college` — your college name
- `$branch` — your engineering branch

Now display them inside an HTML page using Bootstrap cards. Notice how PHP and HTML coexist in the same file — this is server-side templating.

Mission 2 + Challenge



Build a Student Profile Page

Using PHP variables, build a styled `student_profile.php` page. Include: Name, College, Branch, Year of Study, and a short Bio. Use Bootstrap to make it look like a real student ID card. All data must come from PHP variables — no hardcoded HTML text.



Challenge

Add a dynamic **Current Academic Year** field using PHP's `date("Y")` function. If the current month is before June, display the previous year range; otherwise display the current year range. Example: `2024-2025`.

HTML Forms + PHP: The Bridge Between Front & Back

Key Concepts

When a user submits an HTML form, the browser packages the data and sends it to the server. PHP catches it using the `$_POST` superglobal array. Three attributes on your form tag control everything:

- `method="POST"` — sends data in the request body(not the URL)
- `action="process.php"` — which PHP file handles it
- `name="fieldname"` — the key in `$_POST`

```
<form method="POST" action="process.php">
  <input name="student_name">
  <button type="submit">Submit</button>
</form>
```

```
// In process.php:
$name = $_POST['student_name'];
```

Live Coding

Build a Student Registration Form together. Create `register.html` with four fields: Name, Email, Branch, Phone. Set `method="POST"` and `action="process.php"`. Then create `process.php` to receive the data and echo each value back. Open the browser, fill the form, hit Submit — and watch PHP catch the data in real time.

Mission 3 + Challenge

1

Capture Form Data

Build a complete registration form that captures: **Name, Email, Branch,** and **Phone Number**. Style it with Bootstrap. On submission, your `process.php` should display all four submitted values in a formatted summary card.

2

Display Submitted Values

Format the output nicely. Use a Bootstrap card or table to display the submitted student info. Add a heading like "Registration Received" with the student's name dynamically embedded: *"Welcome, Priya!"*

3

Challenge: Reject Empty Name

Add a condition in `process.php`: if the name field is empty, display a red error message and stop processing. Use `empty()` or a string length check. The form should not produce a confirmation if name is blank.

■ 12 MINUTES

MODULE 3 MISSION

Form Validation: Never Trust User Input

Why Validation Matters

Users make mistakes. Some users are malicious. Validation protects your application and your data. Always validate on the **server side** — client-side validation (HTML5 `required`) can be bypassed. PHP gives us powerful tools to validate every field before we process or store anything.

```
$errors = [];  
  
if (empty($_POST['name'])) {  
    $errors[] = "Name is required.";  
}  
  
if (!filter_var($_POST['email'],  
    FILTER_VALIDATE_EMAIL)) {  
    $errors[] = "Invalid email address.";  
}  
  
if (count($errors) > 0) {  
    // Show errors  
} else {  
    // Process data  
}
```

Live Coding — Add Validation

Open your `process.php` from Mission 3. Add an `$errors` array. Validate each field:

- Name: must not be empty
- Email: use `FILTER_VALIDATE_EMAIL`
- Phone: must be 10 digits

If `$errors` has items, display them in a red Bootstrap alert. If all pass, show the green success card. Test with bad data first — watch the errors fire.

Mission 4 + Challenge

Email Validation

Use PHP's built-in `filter_var($email, FILTER_VALIDATE_EMAIL)` to validate the email field. Display a user-friendly error like *"Please enter a valid email address"* — not a raw PHP warning.

Phone Validation

Validate that the phone number is exactly 10 digits. Use `strlen()` and `is_numeric()`. Display an error if the number is too short, too long, or contains letters.

Friendly Errors

Collect ALL validation errors into a single `$errors` array and display them as a bulleted Bootstrap alert box. Don't stop at the first error — show every problem at once so the user can fix everything in one go.



Spot the Bug!

The code below is supposed to process a student registration form — but it's broken. Four bugs are hiding in here. Find them all and write down your fixes before the reveal.

```
<form method="GET" action="process.php">  
  <input type="text" name="student_name">  
  <button type="submit">Register</button>  
</form>
```

```
<?php  
  $name = $_POST['name']  
  echo "Welcome, " . $name  
?>
```



Build Sprint — Student Registration System

This is your main project for today. You have everything you need. Now build it — for real.

1

HTML Form with Bootstrap

Name, Email, Branch, Phone — styled, clean, and responsive

2

PHP Processing Script

Receive all POST fields and echo them back correctly

3

Full Validation

Required fields, valid email, 10-digit phone, friendly error messages

4

Success Confirmation

A Bootstrap success card displaying all submitted student details



★ Bonus: Add a dynamic student profile card UI, better Bootstrap styling, or a multi-section form layout.



Student Showcase

Each team gets **2 minutes** to present. Keep it tight, keep it real. No slides needed — just open your browser and walk us through what you built.

Show the Project

Open your registration form in the browser. Fill it in live. Submit it. Show the confirmation page.



Explain a Challenge

What was the hardest part? A bug you hunted? A validation that wouldn't work? Tell us.

Share the Win

What's the one thing you learned today that clicked? Something that surprised you about PHP?



Interview Quiz — Could You Answer These?

These are real interview questions for junior developer roles. Answer them now — in your own words, out loud, no notes.

1

Frontend vs. Backend

What's the difference? Give a concrete example of each in the context of today's project.

2

GET vs. POST

What's the functional difference? When would you use each? What are the security implications?

3

Purpose of PHP

In one sentence: why do we use PHP?
What can it do that JavaScript in the browser cannot?

4

Purpose of `$_POST`

What is `$_POST`? What type of data structure is it? How do you access a specific field from it?

5

Predict the Output

If a form has `name="city"` and the user types "Pune", what does `echo $_POST['city'];` print?

Today's Assignment

Upgrade the Student Registration System you built today. No new concepts required — just apply what you know more deeply and creatively.



Photo Upload UI

Add a file input field for a profile photo. Style it nicely with Bootstrap. No backend processing needed yet — just the front-end UI and a placeholder display.



More Fields

Add Gender (radio buttons), Course (dropdown), and Address (textarea). Capture and display all of them in your PHP confirmation page.



Better Validation

Improve your validation: no numbers in name, address minimum length, required gender selection. Display all errors together in a styled error box.



Submit your `.php` files and a screenshot of your working form before tomorrow's session starts.

Day 7 Recap

In 2.5 hours, you went from zero PHP knowledge to a working backend application. That's not a small thing.



Modules Completed

Backend, Variables, Forms, Validation



Missions Shipped

One coding mission per module



Full App Built

Student Registration System – end to end

✓ PHP Syntax

Tags, echo, variables, `$_POST`

✓ HTML Forms

method, action, input names

✓ POST Requests

How data travels from browser to server

✓ Validation

Required fields, email, phone, error arrays

Tomorrow: Data Lives Forever

Day 8 — Connecting PHP to MySQL

Browser Sends

PHP Processes

MySQL Stores

Data Persists

Today your application **processed** data. Tomorrow it will **store** data forever.

UP NEXT: DAY 8

PHP + MYSQL